

An immutable file and data store

By Craig Wright | 06 Jan 2019 | Bitcoin & Blockchain Tech

I shall be starting a weekly post on uses of Bitcoin (BSV). BSV allows using the full potential of Bitcoin and the many possible systems and applications that can be created using it. All of the solutions are based on patents we have been granted at nChain; as such, they cannot be used other than on the Bitcoin SV chain.

Alice is in for a ride...

This week, I will cover a system that can be used to create a secure

file store.

Our user, Alice, has an ECDSA public key:

- **Pa(0)** is her public key (it can be registered with a [PKI CA](#)) which is not used as a Bitcoin address. She does not publicly link the "identity key" to her Bitcoin addresses. Rather, she can use the technique in PCT application number [PCT/IB2017/050856](#) to create a deterministic sub key that does link to a used Bitcoin address. We will call it **method 42** hereafter. There are other patents in the entire process, but such is enough for the post.
- **Da(0)** is the *Secret Key* Alice uses to sign messages with Pa(0).
- **Pa(1)** is a deterministic key based on the method above, and is associated with a Bitcoin address. Such an address can be used to hold a file, a contract, an invoice, or even an image safely and securely for as long as the user desires.
- **F(1)** is the first file. It has a hash using a common hash function (such as but not limited to SHA256).

To illustrate how it is possible to build upon such a technique to provide yet further innovations, here is one example of how the technique can be used to secure a file (of any type, but we will use an image file as the example) in a digital wallet (that acts as an app). Public/private key pairs are often used in relation to digital wallets.

In the following example, we have a user who wants to access a file where the file can be encrypted using a separate key for each file. If users are responsible for storing their encryption keys (and the files themselves), problems may arise when the encryption key, the users themselves, or their hardware become unavailable, as one such scenario renders the encrypted files inaccessible. Conversely, though, storage of the encryption key with an application provider requires a degree of trust in the provider and their security

mechanisms. If the app provider's systems are hacked, the encryption key(s) become available to unauthorised parties. Files (such as private images) may then be stolen or compromised. So there is a need to store the encryption key in such a way that it cannot be obtained by an unauthorised party but can also be reproduced when necessary.

In our system, a separate key is used for every file. In general, a single secret key is used for many files; in a standard AES symmetric-encryption-based application, the user will have a single key to protect 1,000s of files. In the system we propose, a separate key is calculated for each file, and yet, the user does not need to fear losing the file nor any of his keys.

A significant benefit of [PCT/IB2017/050856](#) is that use of such a technique allows for the generation of multiple common secrets, corresponding to multiple secure private keys, based on a single private master key for each node. Now, if we take the nodes to be the application "boxes" such that we have a file to be stored, we can use it to create a new key for each file and then allow the file to be stored securely, privately, and permanently on the Bitcoin blockchain.

We achieve such a goal by determining a series of successive deterministic keys on the basis of a process agreed in advance between the application functions. Multiple private keys are consequently kept secure, despite the need to only securely store a single private key at each party. Not only does doing so ease a considerable security burden, it enables the user to generate hierarchies of keys which are derived from a base or master. If the user likes, (s)he can generate "sub-keys," wherein the keys in the hierarchy have a logical association. For example, an operator can generate keys which represent and secure related accounts or entities associated with a particular organisation or individual. Thus,

deterministic keys can be generated securely and in such a manner that they reflect the environment or context in which they are used.

More importantly, when the key and associated Bitcoin address are used to calculate a file address in the Bitcoin blockchain, the file ownership remains pseudonymous. Alice can fund the sending of the file to the deterministic address using a Bitcoin wallet and key that has no relationship to $P_a(0)$ in any way.

We will call such a key and the associated address $P_f(0)$ funding address. Alice can have bitcoin in $P_f(0)$ that has no relation to her identity in any way and still send securely and privately to $P_a(1)$.

In essence, the technique provides for improved secure communication and file storage, and even the creation of secure (and even watermarked) files and exchange between a pair of nodes or parties on a network. A user knows that they can save a file, such as an image, on the Bitcoin blockchain and that it will be available decades later. The technique, with other techniques I will discuss in later posts, allows a user to securely save all the files and data they ever own with no fear of loss or compromise.

The method:

Alice starts with her ECDSA master key (which can be a sub key itself, but I will not overly complicate a complex topic).

- Alice has a file. The parts of the application act as a pair of nodes on a network in our solution and exchange within the application or device (i.e. parties). The application can calculate a key for each file, and allows the user to maintain each file encrypted separately with its own private key and public key and exchange the respective public keys while keeping the

private keys secret.

- The application exchanges a message between its functional components.
- The application can "agree" on a deterministic key which is based on the message. The key is "deterministic" in that the same key will be produced upon multiple executions of the key-generation algorithm.
- Alice has her master key, $Pa(0)$, and a one-time Bitcoin payment address from the public key, $Pf(0)$. For Alice, it does not even matter if the address and key are kept, after she has funded the secure file; in fact, it is best if she uses it for the one payment (and loading of a file) and then discards it. Such is the privacy method used in the Bitcoin white paper.
- Only Alice knows $Pa(0)$, as it is never used on the public blockchain.
- Using the process in **method 42**, Alice creates a secret key $s(af.0)$ between the key she is using to pay miners to host her secure files and the master key. It is then used as the communications key, the process of which is detailed in the patent; it is to secure communications and then to be forgotten.
- The file encryption key $s.f(1)$ means the same process and is detailed below.
- Alice now calculates an address she can easily determine later and that can be calculated in an app using a deterministic process:

$$s(file.1) = H[Da(0) \mid H(file) \mid INDEX]$$

$$Pf(1) = s(file.1) \times G \rightarrow \text{Such is an elliptic-curve operation.}$$

- The index value can be determined in many ways, and is not covered in the post here to maintain some level of simplicity.
- The *Index* can be as simple as:

*INDEX= Hash(index); where **index** is the file number or even a simple non-cryptographic hash or even a simple file checksum.*

- Pa(0) remains private in the entire process. In fact, Pa(0) can even be a threshold-based key such as in [WO2017145010A1](http://patentimages.storage.googleapis.com/e9/d4/1a/644d344019a178/EP3268914B1.pdf).

Method 42—<http://patentimages.storage.googleapis.com/e9/d4/1a/644d344019a178/EP3268914B1.pdf>

The process now allows Alice to send a file from the Bitcoin address associated to Pa(1), where Pa(1) is calculated through a method such as:

- $Pa(1) = Pf(1) + Pa(0) = [s(\text{file.1}) + Da(0)] \times G$

Alice can now send a file using a transaction that incorporates the file she wishes to save to the blockchain. Small files can be sent using either OP_Return or OP_PushData, depending on whether it needs to remain or could be pruned:

- OP_PUSHDATA1 / **{DATA}** / OP_DROP

OP_PushData4 allowed 4.3 GB to be pushed to the stack

The file is signed using ECDSA and thus authenticated. In encrypting (AES and more can be used as a symmetric-encryption algorithm), she knows it was her file, even though she no longer has the private key for Pf(0), as she can compute a secret that is used to decrypt the file using the public information and Pa(0).

The file in the transaction is encrypted using a **method-42** process.

- $Pf(0) = Df(0) \times G$
- $Pa(0) = Da(0) \times G$
- $s.f(1) = Da(0) \times Df(0) \times G = Df(0) \times Pa(0) = Da(0) \times Pf(0)$

As Alice knows $Pa(1)$ and can see that $Pf(0)$ was used to send a file to $Pa(1)$, she can see the public key $Pf(0)$. As a result, and as she knows $Da(0)$, she can calculate $s.f(1)$.

So, Alice can compute the keys used and the file location. The file is encrypted using the symmetric key, $s.f(1)$. So no external party can determine the hash of the file, as they cannot view the file.

Alice can now use a simple hashtable-based system to map many files. With a small amount of information, Alice will be able to access her files securely and from any location and any system. I will explain this aspect later this month.

Unix links files, and uses a simple directory to create a folder. Alice can do the same. With only a key index, she can now start to access her files anywhere.

More importantly, only Alice knows the existence of the files. Even though $Pa(1)$ is derived using a deterministic system from $Pa(0)$, the existence of $Pa(1)$ on the blockchain still gives an external party no way to link the file stores in the transaction $Pa(1)$ to Alice nor even to merely determine a relationship to $Pa(0)$.

As a result, Alice has complete access to all her files. Not just now, but as long as she wants. She can save files from her childhood and come back and find them 50 years later. A system that saves images could save each image once (and only once), as the ability to match hashes means Alice will always know if she has already saved a copy of a file. She will be able to create entire drive stores that are available to her and her alone, and to maintain complete privacy based on pseudonymous linking.

Alice can also create firewalls and partitions. Using the **method-42** process, Alice can now encrypt each file separately, and hence she

can share each file (in whole or part) and even sell access to files. More importantly, unlike a drive-encryption system where all files are encrypted using a single key, Alice has a separate key for all files. If a single key is shared and compromised, it does not endanger the security and privacy of the other files.

As only Alice knows the values in $P_a(1)$ and the file, she has a completely pseudonymous, highly available, and distributed file share. It is private, encrypted, and traceable. She can even use a GIT-like system to map the changes to files over time. As Alice can have a single copy of each file she has and only one copy (that can be accessed anywhere), she uses far less storage than you would expect.

We can make the procedure more secure using HMAC's and other schemes where Alice has even more security and privacy, but doing so is beyond the scope of today's post.

XOR and OTPs

We can even move away from AES and slow encryption. The ultimate encryption system is an OTP (or one-time pad). It can be used with XOR and a Zeta function to create a fast and secure single-use encryption system.

I will not cover such mechanisms in detail in today's post, but it relates to other patents we have coming out during the year.

A two-party system

We now add Bob.

The technique further enables secure communication between the parties, without the need to store the common secret, since the

common secret can be separately determined by each party as required on the basis of the shared message. Importantly, the message does not need to be stored with the same degree of security as the private keys and in some cases may be publicly available.

Our user, Bob, has an ECDSA public key:

- Pb(0) is his public key (it can be registered with a [PKI CA](#)) which is not used as a Bitcoin address. He does not publicly link his "identity" key to his Bitcoin addresses. Rather, he can use the technique in PCT application number [PCT/IB2017/050856](#) to create a deterministic sub key that does link to a used Bitcoin address.
- Db(0) is the *Secret Key* Bob uses to sign messages with Pb(0).
- Pb(1) is a deterministic key based on the method above, and is associated with a Bitcoin address. Such an address can be used to hold a file, a contract, an invoice, or even an image safely and securely for as long as the user desires.

Each node determines:

- an updated version of its own private key, based on its existing private key and the deterministic key, and
- an updated version of the other node's public key, based on the other node's existing public key and the deterministic key

Updated determination may be achieved by applying a neat mathematical process to the existing private key and the deterministic key.

- Each of the pair of nodes then determines a common (i.e. shared) secret on the basis of its own updated private key and the other node's updated public key. As the deterministic key is

based on a shared message, and is therefore common to both nodes, the same common secret can be determined by both nodes, but by means of a combination of different updated private and public keys. The common secret can then be used as the basis for secure communication between the nodes.

Basically, we can use a modified version of the process listed above and incorporate data into a transaction between Alice and Bob.

In such a **permutation**, Alice and Bob each know the secret, and can each access the file and prove it came from the other party. At the same time, even though the file is publicly available, it cannot be decrypted.

Alice and Bob could even allow a transaction to have a pre-signed nLocktime-based expiry so that either party could choose to send the UTXO expire transaction (that is not stored on-chain), allowing the file to expire and be pruned. Again, there is much more to explain, but today's aspects alone are sufficient to spawn several new companies (sorry Dropbox, OneDrive, and Google Drive...)

In conclusion

The applications for [PCT/IB2017/050856](#) are clearly numerous and varied, and are not even limited to use with Bitcoin or blockchain environments. Essentially, such an innovation can provide significant security benefits for any situation in which sensitive data, communications, or controlled resources need to be secured.

Therefore, its potential use cases are countless as the digital world grows with increased cloud storage of data, newer methods of digital communication, and the anticipated explosion of Internet of Things devices.

Welcome to Metanet. The system is deeper than any Rabbit hole you can imagine...

Now, I start to explain it all.

References:

1. **Determining a common secret for the secure exchange of information and hierarchical, deterministic cryptographic keys.**

<https://patentimages.storage.googleapis.com/e9/d4/1a/644d344019a178/EP3268914B1.pdf>

2. Secure multiparty loss-resistant storage and transfer of cryptographic keys for blockchain-based systems in conjunction with a wallet management system

<http://patentimages.storage.googleapis.com/38/81/de/27b37646a28b52/WO2017145010A1.pdf>

This story is published in [The Startup](#), Medium's largest entrepreneurship publication followed by +411,714 people.

Subscribe to receive [our top stories here](#).